Skillo AI: An Uncertainty-Calibrated, Tree-Search-Driven Mock Interview System with GraphRAG Skill-Gap Mapping

Shakila Shaikh
Assistant Professor
[Department of ...]
[Institute name]
[City, State, Country]
[email@domain]

Ritunjay Deo
[Programme] Student
[Department of ...]
[Institute name]
[City, State, Country]
[email@domain]

Yug Dhaigude
[Programme] Student
[Department of ...]
[Institute name]
[City, State, Country]
[email@domain]

Sylborn Furtado
[Programme] Student
[Department of ...]
[Institute name]
[City, State, Country]
[email@domain]

***Abstract*—Interview-preparation tools still rely on fixed question scripts, keyword résumé matching and single-pass language-model feedback, which gives uncalibrated scores, no adaptive probing and no memory across sessions. We present Skillo AI, a full-stack mock-interview and résumé-intelligence platform that combines five recent research ideas: Prometheus-2-style rubric scoring with Semantic Entropy confidence, Language Agent Tree Search (LATS) for adaptive follow-up questions, GraphRAG skill-gap mapping with prerequisite chains, Reflexion-style verbal memory, and a SimPO-inspired contrastive benchmark comparison. An evidence-grounding verifier and an on-device delivery module (speech, gaze, posture, affect) complete the pipeline. The system is implemented in Next.js and TypeScript behind authenticated, validated API routes and is covered by 615 automated tests. In a live pilot on six pairs from a 50-pair benchmark corpus, an open-weight 120B model ranked the benchmark answer above the weak answer in 6/6 pairs (mean gap +1.08 on a 1-5 scale; Wilson 95% CI for accuracy 0.61-1.00), but every evaluation received HIGH confidence, so calibration could not be assessed. Scoring takes 5.9 s for a two-answer submission, deployable on-device vision models run in 6.8-8.4 ms per frame, and we show analytically that the original length-normalised SimPO margin could never reach its threshold on our corpus. Results are preliminary; larger human-labelled calibration and user studies remain future work.**

***Index Terms*—mock interview simulation, large language models, semantic entropy, tree search, knowledge graph**

I. Introduction

A. Background

Software-engineering hiring combines coding rounds, system-design discussions and behavioural interviews, and candidates usually prepare alone. Large language models (LLMs) can now both pose and grade interview questions, and LLM judges agree reasonably well with human preferences on open-ended answers [1]. This makes automated, always-available interview practice technically feasible, but a useful coach must do more than return a number: it must say how sure it is, probe weaknesses, connect gaps to missing prerequisites, and remember what the candidate struggled with last time.

B. Problem Statement and Existing Limitations

Existing preparation tools fall into three groups. Algorithm platforms check test-case outcomes but give no feedback on trade-offs or communication. Generic chat wrappers answer in a single pass, with scores that vary between runs and no stated uncertainty. Applicant-tracking-style résumé tools match keywords and ignore that a missing foundational skill (for example concurrency primitives) blocks an advanced one (distributed systems design). None of them keeps structured memory across sessions or shows how an answer differs from a strong reference answer.

C. Research Gap and Motivation

Recent work supplies the individual pieces: rubric-conditioned evaluators [2], uncertainty estimation by semantic entropy [3], tree search over language-model actions [4], graph-based retrieval and community structure [5], verbal reinforcement across trials [6], and reference-free length-normalised preference rewards [7]. To our knowledge these have not been composed into one interview-coaching pipeline whose behaviour is checked end to end, including the failure modes that appear when the pieces meet real model output, rate limits and noisy input.

D. Objectives and Contributions

The objective is a deployable system, with measured behaviour, that scores answers with a stated confidence, adapts its questions, maps skill gaps to prerequisites and tracks progress. The main contributions are:

- an end-to-end architecture that integrates the five engines above with an evidence-grounding verifier and a multimodal delivery module behind authenticated, validated, rate-limited API routes;
- honest-by-construction reporting: every result carries a live/fallback flag, offline estimates are capped and marked, and a result is never labelled live unless real model calls succeeded;

- an analysis showing that a length-normalised reward margin with a fixed threshold is unreachable on our benchmark corpus, and a quality-margin replacement;
- a live pilot, latency measurements and an engineering audit (615 automated tests) that document where the system works and where it does not.

II. Related Work

A. Existing AI Methods and Recent Studies

LLM evaluation. Prometheus 2 merges direct-assessment and pairwise-ranking evaluators and scores against user-supplied rubrics [2]; it judges one output in isolation. **Uncertainty.** Semantic entropy clusters sampled answers by meaning and measures the entropy of the cluster distribution to detect confabulation [3]; it was studied for factual question answering rather than multi-criterion rubric grading. **Planning.** LATS combines Monte Carlo tree search with a value function and reflection over LLM actions [4], [8], [9], at the cost of many model calls.

Retrieval and memory. GraphRAG builds an entity graph and community summaries for corpus-level questions [5], extending retrieval-augmented generation [10]; Self-RAG [11] and Corrective RAG [12] add self-critique and retrieval-quality checks. Reflexion stores verbal self-critiques to improve later attempts without weight updates [6], but its memory is episodic and per task. **Preference and layout.** SimPO uses an average-log-probability reward with a target margin [7]; LayoutLMv3 jointly models text and page layout for document understanding [13].

TABLE I
CAPABILITY COMPARISON OF INTERVIEW-PREPARATION APPROACHES

Capability	Algorithm sites	Chat wrappers	ATS tools	Skillo
Rubric-based scoring	No	Partial	No	Yes
Stated score confidence	No	No	No	Yes
Adaptive follow-ups	No	Partial	No	Yes
Prerequisite skill graph	No	No	No	Yes
Cross-session memory	No	No	No	Yes*
Benchmark answer delta	No	No	No	Yes
Voice and delivery signals	No	No	No	Yes

*Qualitative design comparison from the project's literature review, not a measured benchmark. *Memory persistence is implemented against a Supabase JSONB store.*

B. Identified Research Gap

Individual methods are evaluated in isolation, on factual or code benchmarks, with synthetic or offline conditions. What is missing is an integrated, deployed coach whose scoring, uncertainty, adaptivity and memory are exercised together on live model output, with explicit reporting of when the system silently degrades.

III. Proposed Methodology

A. Overall Architecture

Fig. 1 shows the pipeline. The browser captures a résumé, the spoken or typed answers, optional code and a system-design whiteboard, plus camera and microphone streams. Server route handlers authenticate the caller (Firebase ID token or Supabase session), validate input with Zod schemas, apply per-user rate limits and call the engines. The interview cannot start until a live camera and microphone are present, and the question is read aloud with the browser speech-synthesis API.

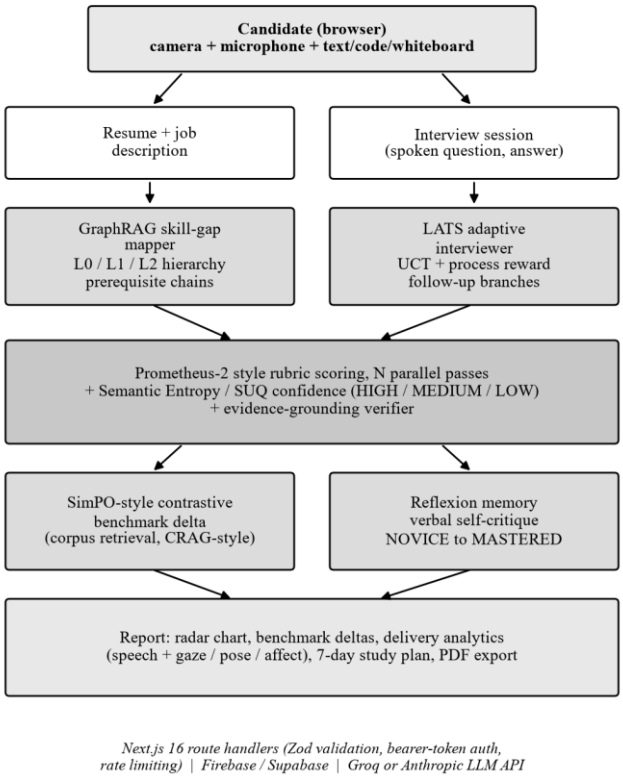


Fig. 1. System architecture of Skillo AI.

B. Dataset

We assembled a benchmark corpus of 50 interview questions, each with a strong benchmark answer (*preferred*) and a weak answer (*dispreferred*): 15 frontend architecture, 10 distributed systems, 10 system design, 8 ML engineering and 7 behavioural (STAR). The corpus is split into 35 calibration, 7 validation and 8 test pairs. Preferred answers average 76.0 words and dispreferred answers 23.1 words, a length difference that matters for the analysis in Section VI. The corpus was assembled within the project, is not a public benchmark, and its answers have not been independently validated by human raters.

C. Data Preprocessing

All user text passes a sanitiser that strips markup, control and zero-width characters, wraps known prompt-injection phrases in a marker instead of deleting them, collapses whitespace and truncates (5,000 characters per answer, 15,000 per résumé). Résumés are parsed server-side from PDF, DOCX or text (5 MB limit, content-type check). Token length is approximated as 1.3 times the word count. Entity names are normalised so that variants of the same technology map to one node. Camera frames are cleaned by white balance, denoising, a blur gate (variance of the Laplacian, threshold 100) and illumination normalisation before inference.

D. Rubric Scoring and Semantic Entropy

An answer is scored by N=3 parallel passes (N=5 in deep mode) at temperature 0.7 against four weighted criteria (technical accuracy, system architecture and logic, edge-case awareness, communication) on a 1-5 scale. Each pass must justify every criterion with a claim and a verbatim quote from the answer. The overall score of a pass is

$$S = \sum_{k=1}^{4} w_k \, s_k, \quad \mathbf{w} = (0.35,\ 0.30,\ 0.20,\ 0.15) \tag{1}$$

Passes are clustered by meaning with one temperature-0 model call that returns a partition of the pass indices (bidirectional entailment); the partition is validated and, if the call fails, passes are clustered by score difference (at most 0.5). With cluster probabilities $P(c)=|c|/N$ the semantic entropy and confidence tier are

$$\mathrm{SE}(x) = -\sum_{c \in C} P(c) \log_2 P(c), \quad P(c) = \tfrac{|c|}{N} \tag{2}$$

$$\mathrm{conf} = \mathrm{HIGH}\ (\mathrm{SE} \le 0.3),\ \mathrm{MEDIUM}\ (\le 0.8),\ \mathrm{LOW}\ (> 0.8) \tag{3}$$

The final score is the probability-weighted mean of cluster scores. A grounding verifier checks each quote against the answer (fuzzy match, 0.85 threshold); passes with fewer than half of their quotes verified are excluded from the entropy computation. Offline heuristic passes are flagged, capped at 3.0 and always receive LOW confidence.

E. Adaptive Follow-up by Tree Search

After each answer, a bounded LATS search expands the current node into three branches (deep-dive, pivot, edge-case challenge), scores each with a process-reward prompt in [0,1] (temperature 0.2) and selects by UCT, with unvisited nodes first and a uniform prior over siblings:

$$\mathrm{UCT}(s,a) = Q(s,a) + c_{\mathrm{puct}} \, P(s,a) \, \tfrac{\sqrt{N(s)}}{1+N(s,a)}, \quad c_{\mathrm{puct}} = 1.414 \tag{4}$$

$$Q_{\mathrm{new}}(s) = \tfrac{n(s)\,Q_{\mathrm{old}}(s) + v}{n(s)+1}, \quad v \in [0,1] \tag{5}$$

The number of rounds defaults to 3 with Anthropic models and 1 on the rate-limited free Groq tier; a wall-clock budget of 10 s bounds the search. The tree is returned to the client and sent back on the next turn (at most 100 KB and depth 6), so the trajectory accumulates; one follow-up is allowed per question.

F. GraphRAG Skill-Gap Mapping

An LLM extracts typed entities (skill, framework, concept, domain) and five relation types (depends-on, applied-in, expands-upon, requires, blocks) from the résumé and job description. Label drift is tolerated by coercing unknown types to the nearest known one. Connected components are found by breadth-first search; a node's level is assigned from topology (no incoming edge: macro domain L0; incoming and outgoing: pillar L1; incoming only: leaf L2), and prerequisite chains are produced by walking edges upward from missing leaves with cycle protection.

G. Reflexion Memory and Contrastive Benchmark Delta

After the response is returned, a non-blocking worker writes a verbal self-critique (skill, mistake, root cause, remediation, severity). Progress for a skill over its last five critiques is $(100\ n_{\mathrm{low}} + 50\ n_{\mathrm{med}})/n$, mapped to NOVICE, DEVELOPING (≥40), PROFICIENT (≥65) and MASTERED (≥85 with at least three attempts). For benchmark comparison, a TF-IDF retriever finds the closest corpus question; similarity ≥0.45 uses the stored benchmark answer, 0.25-0.45 refines it with the model, and lower values generate one. The original SimPO-style reward and our replacement margin are

$$r(x,y) = \tfrac{\beta\, q(y)}{\max(1,|y|)}, \quad |y| = \lceil 1.3 \cdot \mathrm{words}(y) \rceil, \ \beta = 2 \tag{6}$$

$$\Delta q = q_{\mathrm{pref}} - q_{\mathrm{cand}} \ \ge\ \gamma = 0.25 \tag{7}$$

H. Multimodal Delivery Module

Speech analytics (words per minute, filler words, pauses) are computed from recognition timestamps. A browser worker runs MediaPipe Face Landmarker [14] and lightweight ONNX models for gaze, head pose and affect, inspired by [15], [16], [17]; lip-audio synchrony [18] is reduced to a rule-based audio-presence indicator because the two-stream model is far over the frame budget. Signals are smoothed and fused, with weights renormalised over the available signals:

$$S_t = \alpha X_t + (1-\alpha)\, S_{t-1}, \qquad J = \tfrac{1}{T-1} \sum_{t=2}^{T} |X_t - X_{t-1}| \tag{8}$$

$$D = \tfrac{\sum_{i \in P} w_i x_i}{\sum_{i \in P} w_i}, \quad \mathbf{w} = (0.30,\ 0.25,\ 0.20,\ 0.25) \tag{9}$$

The three vision models use a MobileNetV2 backbone [19] and are trained for 20 epochs with AdamW [20] (learning rate 10^{-4}, weight decay 10^{-2}, cosine annealing), batch size 8, seed

42 and 224×224 inputs with ImageNet normalisation. Pose uses 3DDFA-V2 and BIWI [21], [22] with SmoothL1 loss, affect uses FER2013 [23] with cross-entropy, and gaze uses MPIIGaze [24] with SmoothL1 loss; augmentation is brightness/contrast jitter and a 50% horizontal flip. The LLM components are used zero-shot with prompts and schemas, without fine-tuning.

I. Evaluation Metrics

For the scorer we report pairwise accuracy (fraction of pairs in which the benchmark answer scores higher), the paired score gap, the confidence-tier distribution and expected calibration error with ten bins [25], with Wilson intervals [26] for proportions. For systems we report end-to-end latency (mean, standard deviation) and the per-frame budget for vision. For training we report loss curves.

IV. Experimental Setup

A. Hardware and Software

Live experiments ran on a laptop with an Intel Core i5-13450HX (10 cores, 16 threads), 15.7 GB RAM and an NVIDIA GeForce RTX 3050 6 GB Laptop GPU under Windows 11, calling a hosted API over the public internet. The application uses Node.js v26.3.0, Next.js 16.2, React 19.2, TypeScript 5.8, Zod 4.4 and Jest 30.4; scripts and figures use Python 3.12. The scoring model is the open-weight gpt-oss-120b [27] served by Groq with reasoning effort set to low.

B. Dataset Split and Parameters

The pilot uses the first three pairs of the validation split (fe_001-fe_003) and of the test split (fe_006-fe_008), six pairs in total. This limit comes from the provider's free-tier request and token limits, not from the design of the harness, which accepts the full corpus. Table II lists the main parameters.

TABLE II
Main system parameters

Parameter	Value
Scoring passes N / temperature	3 (5 deep) / 0.7
Criterion weights	0.35, 0.30, 0.20, 0.15
Entropy tiers	≤0.3 HIGH, ≤0.8 MEDIUM
Clustering call / grounding threshold	temp. 0 / 0.85
LATS c_{puct}, rounds, budget	1.414, 1 (Groq) or 3, 10 s
SimPO β, γ (quality margin)	2.0, 0.25
Retrieval thresholds	0.45 / 0.25
Telemetry cap per stream	1,000 frames
Max concurrent model calls (Groq)	2 (1 in pilot, latency runs)
Per-user limits (per minute)	evaluate 5, follow-up 10

C. Baselines and Evaluation Protocol

The comparison targets named in the project plan are a single-pass LLM scorer and a keyword résumé matcher. Both are available in the code (N=1 and an offline heuristic), but we did not run head-to-head experiments against them because of the API budget; Table I therefore compares capabilities, not accuracy. The protocol for the pilot is: evaluate the preferred and dispreferred answer of each pair in scoring-only mode, sequentially, and retry with back-off whenever a result is not fully live (in practice, throttled model calls), so that only fully live results are kept and fallback results are never reported. Latency is wall-clock time of the whole server function with one concurrent model call, including the deliberate retry waits.

V. Results

A. Scoring Pilot

The scorer ranked the benchmark answer higher in 6 of 6 pairs (accuracy 1.00, Wilson 95% interval 0.61-1.00). The mean score was 3.72 (SD 0.15) for benchmark answers and 2.64 (SD 0.33) for weak answers, a mean paired gap of +1.08 (SD 0.26, minimum +0.60). Fig. 2 and Table III give the per-pair scores.

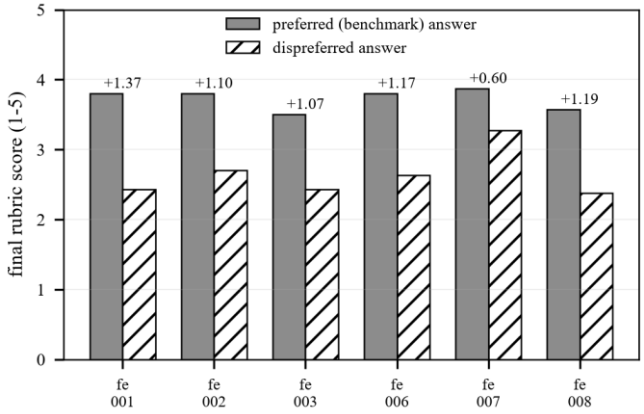


Fig. 2. Final rubric score for the benchmark and weak answer of each pilot pair (gap above bars).

TABLE III
Per-pair pilot scores (1-5 scale)

Pair	Benchmark	Weak	Gap
fe-001	3.80	2.43	+1.37
fe-002	3.80	2.70	+1.10
fe-003	3.50	2.43	+1.07
fe-006	3.80	2.63	+1.17
fe-007	3.87	3.27	+0.60
fe-008	3.57	2.38	+1.19
Mean	3.72	2.64	+1.08

All 12 evaluations fell in the HIGH tier with semantic entropy 0, because the three passes of every evaluation landed in a single meaning cluster. Consequently expected calibration error is 0.00 on both splits, which is uninformative: the confidence is constant and the accuracy is 1. A confusion matrix for the tier-versus-correctness decision is therefore

trivially diagonal (12 of 12 HIGH, all pairs correct) and we do not tabulate it. Three further scoring-only runs on a two-question submission gave mean grounding 0.97 (range 0.92-1.00), semantic clustering in all runs, and entropy 0.

B. Latency

TABLE IV
MEASURED LATENCY OF SERVER FUNCTIONS (GROQ, GPT-OSS-120B)

Function	Runs	Mean (s)	Individual runs (s)
Scoring only (3 passes + clustering)	3	5.9	5.3, 6.8, 5.4
Full evaluation (+ SimPO + LATS)	2	38.0	40.2, 35.7
Follow-up turn (1 search round)	3	10.8	11.5, 11.3, 9.4
Résumé graph extraction	2	9.1	7.8, 10.3

These times were taken with a single concurrent model call and are therefore upper-bounds; a single earlier run of one search round with the default limit of two concurrent calls took 2.3 s. The follow-up measurements (10.8 s) are close to the 12 s the interview screen waits for a follow-up, so the concurrency limit matters in practice. Two graph extractions of the same résumé and job description produced 26 and 32 nodes and an overall coverage of 4% and 28%, showing that the coverage figure is unstable between runs.

C. Multimodal Delivery Module

Fig. 3 shows normalised training and validation loss of the three vision models, summarised in Table V. Head-pose validation loss improves until epoch 12 (40.57 to 35.92); gaze validation loss is lowest at epoch 7 and then rises; affect validation loss is lowest at epoch 1 and rises while training loss falls to 0.79, a clear overfit. The best affect validation cross-entropy (1.96) is not below the chance level for seven classes (ln 7 = 1.95). The affect model is therefore only a weak auxiliary signal in the system and we do not report held-out accuracy: the datasets are not distributed with the repository and could not be re-run for this paper.

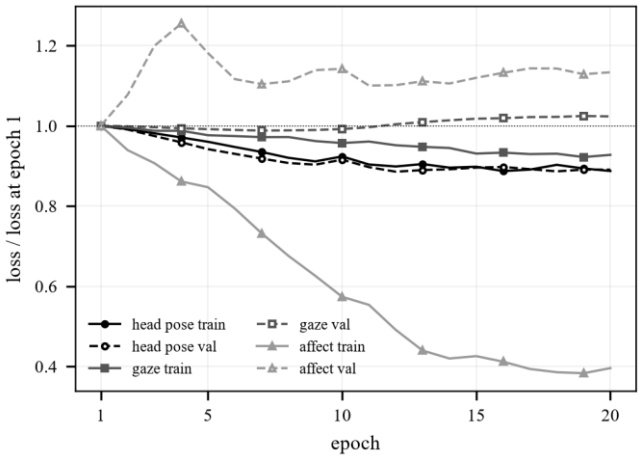


Fig. 3. Normalised training (solid) and validation (dashed) loss over 20 epochs.

TABLE V
TRAINING SUMMARY OF THE VISION MODELS

Model	Best val epoch	Best val	Final train / val
Head pose	12	35.92	33.64 / 36.13
Gaze	7	13.41	10.18 / 13.89
Affect	1	1.96	0.79 / 2.23

Fig. 4 shows the steady-state per-frame latency measured by the project's browser benchmark harness. MediaPipe Face Landmarker (6.8 ms) and the affect classifier (8.4 ms) fit the 15 ms budget, whereas the full gaze (28.5 ms) and head-pose (22.1 ms) networks and the SyncNet two-stream model (72.7 ms) do not; the latter was therefore replaced by rules. The preprocessing stages add 1.0 ms live (6.8% of the budget). Exponential smoothing reduced frame-to-frame jitter by 26.1% (head yaw), 35.7% (gaze pitch), 81.4% (valence) and 83.7% (composure), Fig. 5, and the total per-frame pipeline cost was measured at 8.68 ms.

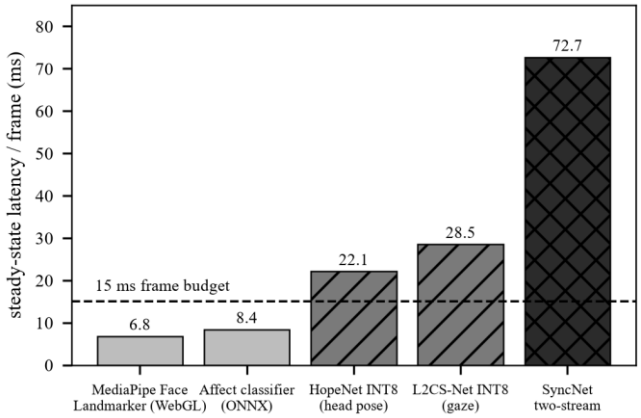


Fig. 4. Steady-state latency per frame of candidate vision models against the 15 ms budget.

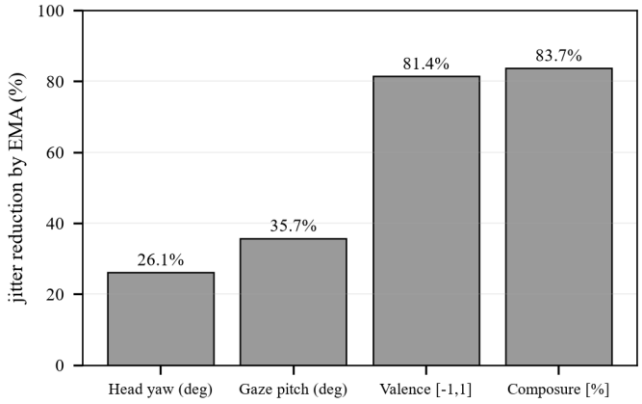


Fig. 5. Jitter reduction obtained by exponential moving-average smoothing.

D. Software Quality

The code base has about 39,000 lines of TypeScript, seven API routes and 76 test files with 615 test cases (Jest), all passing, and a clean type check and production build at the time of writing. Tests cover the engines, request validation, authentication, rate limiting, fallback honesty, speech analytics, the device gate and the study-plan logic.

VI. Discussion

A. Why the Scorer Separates the Answers, and What the Pilot Cannot Show

The scorer gave a consistent gap of about one point (min +0.60), which is the expected direction because the benchmark answers name mechanisms, complexity and failure modes. However, benchmark answers are about 3.3 times longer than weak ones, so part of the gap may come from length rather than quality; a length-controlled set is needed. With six pairs the interval for accuracy remains wide (0.61-1.00), only one model and one provider were used, and no human ratings were collected, so the results show feasibility, not validity.

B. Confidence Collapse

Semantic entropy was exactly 0 in every evaluation. With three samples from a strong model at temperature 0.7 the passes almost always agree on the assessment, so the tiers never separate easy from ambiguous answers. Calibration claims therefore need harder, more ambiguous answers, more samples (deep mode uses five), and human labels.

C. Comparison with Existing Approaches

Against algorithm platforms, generic chat wrappers and keyword résumé tools (Table I), Skillo adds a stated confidence, evidence-checked feedback, adaptive probing, a prerequisite view of gaps and cross-session memory. The cost is complexity and latency: a full evaluation took 38 s with one concurrent call, versus a single model call for a wrapper. We did not measure whether these additions improve scoring accuracy or candidate outcomes against a single-pass baseline, so the advantage is a design difference, not an empirical result.

D. Ablation: Reward Margin

The original SimPO-style margin r_{pref} - r_{cand} divides a quality term by answer length. On our corpus, with a benchmark quality of 0.85, benchmark rewards average 0.018 and weak-answer rewards (quality 0.4) average 0.027; the margin is positive for only 2% of pairs and never reaches the original threshold of 0.5 for any candidate quality (Table VI). The quality margin of Eq. (7) is satisfied for candidate quality up to 0.6 and is not penalised by the length of the benchmark answer. This is an analytic check on the real corpus with assumed qualities, not an experiment with human ratings.

TABLE VI
Reward margin on the 50-pair corpus (benchmark quality 0.85)

Cand. q	Mean r pref	Mean r cand	Old margin >0	Old ≥0.5	Δq
0.2	0.018	0.014	88%	0%	0.65
0.4	0.018	0.027	2%	0%	0.45
0.6	0.018	0.041	0%	0%	0.25
0.8	0.018	0.055	0%	0%	0.05

E. Failure Modes Found During Evaluation

Running the engines on live model output exposed problems that unit tests with mocks had not: (i) a 4.5 s default timeout was applied to the résumé graph extraction and question generation, silently replacing real analyses with a fixed template; (ii) a single entity label outside the schema (for example "LANGUAGE") rejected a whole valid graph, which label coercion fixed (the same inputs then produced a 26-40 node graph); (iii) a configured but failing provider was reported as "live" with HIGH confidence, which we corrected so that mode, confidence and score cap reflect the number of real passes; and (iv) real interview telemetry (about 10 frames per second) exceeded the evaluation payload limits, so telemetry is now sampled per question.

F. Strengths and Limitations

Strengths are the integration, the explicit live/fallback accounting, bounded cost per call, and the on-device delivery signals with measured latency. Limitations are: a small, project-assembled corpus and a six-pair pilot; no human ratings, no length control and no baseline experiments; skill-gap node status derives from graph connectivity (two or more relations) rather than from a direct comparison of résumé and job skills, which makes coverage figures unreliable and unstable; an affect model that does not beat chance on validation loss; memory persistence tied to one storage provider; and reliance on a third-party hosted model whose names and limits change. Candidate video is processed on the device and the system is designed to be non-accusatory, but

bias and fairness of LLM judging across candidate backgrounds have not been studied.

VII. Conclusion

We presented Skillo AI, an integrated mock-interview and résumé-intelligence system that combines rubric scoring with semantic-entropy confidence, tree-search follow-ups, GraphRAG skill-gap mapping, Reflexion memory and a contrastive benchmark comparison, with on-device delivery analysis. A live six-pair pilot showed the scorer separating benchmark from weak answers in all pairs (mean gap +1.08), while also showing that confidence tiers were uninformative and that skill-gap coverage is unstable. Measured latencies are practical for interactive use when follow-up search is limited, and an analytic study replaced a reward margin that could not be satisfied. The main contribution is the integrated, instrumented pipeline and the documented failure modes; the main limitations are the small pilot, the absence of human ratings and baselines, and the unstable skill-gap coverage.

Future work: (1) a larger, human-labelled and length-controlled benchmark with calibration on harder answers and more samples; (2) a head-to-head comparison with single-pass scoring and keyword résumé matching; (3) a skill-gap status that compares résumé and job skills directly; (4) held-out evaluation and, if needed, replacement of the affect model; (5) provider-independent memory persistence; and (6) a user study measuring whether the feedback improves later interview performance.

References

[1] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2023.

[2] S. Kim *et al.*, "Prometheus 2: An open source language model specialized in evaluating other language models," in *Proc. EMNLP*, 2024, arXiv:2405.01535.

[3] L. Kuhn, Y. Gal, and S. Farquhar, "Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation," in *Proc. ICLR*, 2023.

[4] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, "Language agent tree search unifies reasoning, acting, and planning in language models," in *Proc. ICML*, 2024.

[5] D. Edge *et al.*, "From local to global: A graph RAG approach to query-focused summarization," arXiv:2404.16130, 2024.

[6] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. NeurIPS*, 2023.

[7] Y. Meng, M. Xia, and D. Chen, "SimPO: Simple preference optimization with a reference-free reward," in *Proc. NeurIPS*, 2024.

[8] C. B. Browne *et al.*, "A survey of Monte Carlo tree search methods," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1-43, 2012.

[9] L. Kocsis and C. Szepesvári, "Bandit based Monte-Carlo planning," in *Proc. ECML*, 2006, pp. 282-293.

[10] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.

[11] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. ICLR*, 2024.

[12] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," arXiv:2401.15884, 2024.

[13] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "LayoutLMv3: Pre-training for document AI with unified text and image masking," in *Proc. ACM Multimedia*, 2022.

[14] C. Lugaresi *et al.*, "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019.

[15] A. A. Abdelrahman *et al.*, "L2CS-Net: Fine-grained gaze estimation in unconstrained environments," arXiv:2203.03339, 2022.

[16] N. Ruiz, E. Chong, and J. M. Rehg, "Fine-grained head pose estimation without keypoints," in *Proc. IEEE CVPR Workshops*, 2018.

[17] A. Mollahosseini, B. Hasani, and M. H. Mahoor, "AffectNet: A database for facial expression, valence, and arousal computing in the wild," *IEEE Trans. Affect. Comput.*, vol. 10, no. 1, pp. 18-31, 2019.

[18] J. S. Chung and A. Zisserman, "Out of time: Automated lip sync in the wild," in *Proc. ACCV Workshops*, 2016.

[19] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF CVPR*, 2018, pp. 4510-4520.

[20] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.

[21] J. Guo, X. Zhu, Y. Yang, F. Yang, Z. Lei, and S. Z. Li, "Towards fast, accurate and stable 3D dense face alignment," in *Proc. ECCV*, 2020.

[22] G. Fanelli, M. Dantone, J. Gall, A. Fossati, and L. Van Gool, "Random forests for real time 3D face analysis," *Int. J. Comput. Vis.*, vol. 101, no. 3, pp. 437-458, 2013.

[23] I. J. Goodfellow *et al.*, "Challenges in representation learning: A report on three machine learning contests," *Neural Netw.*, vol. 64, pp. 59-63, 2015.

[24] X. Zhang, Y. Sugano, M. Fritz, and A. Bulling, "Appearance-based gaze estimation in the wild," in *Proc. IEEE CVPR*, 2015, pp. 4511-4520.

[25] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, 2017, pp. 1321-1330.

[26] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209-212, 1927.

[27] OpenAI, "gpt-oss-120b and gpt-oss-20b model card," arXiv:2508.10925, 2025.